

## Assessment Tool 3 – Technical Presentation Answer (CO5)

### Topic 5: Multi-Agent System for Disaster Management

#### 1. Technical Content — What is a Multi-Agent System (MAS)?

A **Multi-Agent System** consists of several autonomous agents, each with a specific role, that **perceive, decide, and act independently** but **coordinate** to achieve a shared goal — in this case, detecting a disaster (flood, earthquake, wildfire) early and coordinating an effective response. No single agent has full global knowledge; intelligence emerges from their collaboration and communication.

**Agent roles in this system:**

- **Sensor Agents:** deployed logically per data source (weather sensors, seismographs, river-level sensors, satellite feeds, social-media stream listeners). Each continuously perceives its environment and reports readings/events.
- **Decision Agents:** goal-based/utility-based agents that fuse data from multiple sensor agents, evaluate disaster likelihood/severity against thresholds or learned models, and decide whether to raise an alert and at what severity level.
- **Communication Agents:** responsible for propagating verified alerts to relevant stakeholders (emergency services, local authorities, public alert systems) and for inter-agent messaging (so Decision Agents in different regions can share correlated evidence, e.g., an upstream flood sensor informing a downstream region's decision agent).

#### 2. Problem Identification

Natural disasters unfold across large geographic areas with heterogeneous, high-volume, high-velocity data (satellite imagery, IoT sensor streams, social media posts). A centralized single-agent design becomes a bottleneck and a single point of failure, and cannot process the volume/variety of data fast enough for early warning. The MAS approach decomposes the problem: sensor agents handle local perception and pre-filtering close to the data source, decision agents handle regional fusion and inference, and communication agents handle timely dissemination — the workload is distributed instead of centralized, using PySpark to process each data stream at scale in parallel.

#### 3. System Design & Use of Tools

**Distributed data pipeline (PySpark/RDD):**

- Each sensor stream is ingested as a separate RDD/DStream partition (sensor\_type as key).
- **map:** normalize raw readings into a common event schema (location, timestamp, metric, value, source\_type).
- **filter:** discard noise/out-of-range sensor readings.
- **reduceByKey** (keyed by geographic grid cell): aggregate readings from multiple sensor types within the same region for fusion.
- **join:** correlate satellite-image-derived flood extent estimates with ground sensor readings and social-media event mentions for the same region/time window.
- **mapPartitions:** run the Decision Agent's severity-scoring model per partition (per region), so many

regions are evaluated concurrently across the cluster. - **broadcast**: distribute a small reference dataset (e.g., population density per region, evacuation-route map) to all worker nodes efficiently for use in every regional decision.

#### 4. Coordination Strategies

- **Publish–subscribe messaging**: Sensor Agents publish readings to region-specific topics; Decision Agents subscribe to their region plus adjacent regions (to catch cross-boundary events, e.g., a river flood moving downstream).
- **Escalation protocol**: if a Decision Agent's confidence is below a threshold, it requests corroboration from neighboring Decision Agents before triggering a Communication Agent alert — reducing false alarms.
- **Consensus / voting**: for borderline cases, multiple Decision Agents (or multiple models within one) vote before an alert is finalized, improving robustness against single-sensor failure or noisy data.

#### 5. Real-Time Decision-Making

Sensor-to-alert latency is minimized by: streaming ingestion (Spark Structured Streaming micro-batches instead of batch jobs), regional partitioning (each partition/region processed independently and in parallel), and a tiered alert system (low-confidence “watch” vs high-confidence “warning”) so early signals reach authorities without waiting for full certainty, while still avoiding needless panic from single noisy readings.

#### 6. Suggested Presentation Structure

1. Motivation: why disaster response needs speed + scale
2. MAS overview and the three agent roles (PEAS description for each)
3. Distributed architecture diagram: sensor agents → decision agents → communication agents
4. PySpark RDD operations mapped to each pipeline stage
5. Coordination/consensus mechanism between decision agents
6. Example scenario walkthrough (flood detection end-to-end)
7. Fault tolerance: what happens if a regional node fails (Spark RDD lineage recovery, other agents continue independently)
8. Conclusion and extensions (adding a Response-Planning Agent for resource allocation)

#### 7. Summary

This Multi-Agent System distributes disaster detection across specialized Sensor, Decision, and Communication agents that coordinate via publish–subscribe messaging and consensus protocols, while

PySpark's RDD operations (map, filter, reduceByKey, join, mapPartitions, broadcast) enable the underlying multi-source data fusion to scale across large geographic areas in near real time.